Capstone Project

March 12, 2026

1 Capstone Project

1.1 Probabilistic generative models

1.1.1 Instructions

In this notebook, you will practice working with generative models, using both normalising flow networks and the variational autoencoder algorithm. You will create a synthetic dataset with a normalising flow with randomised parameters. This dataset will then be used to train a variational autoencoder, and you will used the trained model to interpolate between the generated images. You will use concepts from throughout this course, including Distribution objects, probabilistic layers, bijectors, ELBO optimisation and KL divergence regularisers.

This project is peer-assessed. Within this notebook you will find instructions in each section for how to complete the project. Pay close attention to the instructions as the peer review will be carried out according to a grading rubric that checks key parts of the project instructions. Feel free to add extra cells into the notebook as required.

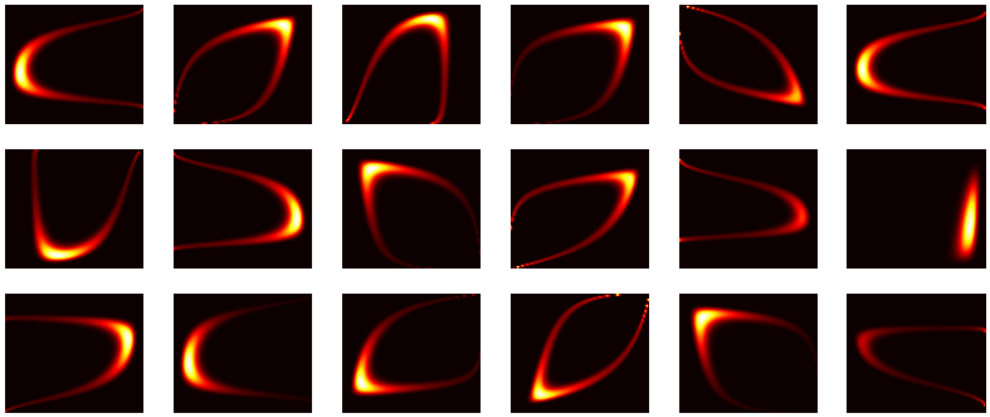
1.1.2 How to submit

When you have completed the Capstone project notebook, you will submit a pdf of the notebook for peer review. First ensure that the notebook has been fully executed from beginning to end, and all of the cell outputs are visible. This is important, as the grading rubric depends on the reviewer being able to view the outputs of your notebook. Save the notebook as a pdf (File -> Download as -> PDF via LaTeX). You should then submit this pdf for review.

1.1.3 Let's get started!

We'll start by running some imports below. For this project you are free to make further imports throughout the notebook as you wish.

In [10]:
```
import tensorflow as tf
import tensorflow_probability as tfp
tfd = tfp.distributions
tfb = tfp.bijectors
tfpl = tfp.layers

import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```



Flags overview image

For the capstone project, you will create your own image dataset from contour plots of a transformed distribution using a random normalising flow network. You will then use the variational autoencoder algorithm to train generative and inference networks, and synthesise new images by interpolating in the latent space.

The normalising flow

- To construct the image dataset, you will build a normalising flow to transform the 2-D Gaussian random variable $z = (z_1, z_2)$, which has mean $\mathbf{0}$ and covariance matrix $\Sigma = \sigma^2 \mathbf{I}_2$, with $\sigma = 0.3$.
- This normalising flow uses bijectors that are parameterised by the following random variables:
  - $\theta \sim U[0, 2\pi)$
  - $a \sim N(3, 1)$

The complete normalising flow is given by the following chain of transformations: * $f_1(z) = (z_1, z_2 - 2)$, * $f_2(z) = (z_1, \frac{z_2}{2})$, * $f_3(z) = (z_1, z_2 + a z_1^2)$, * $f_4(z) = Rz$, where R is a rotation matrix with angle θ, * $f_5(z) = \tanh(z)$, where the tanh function is applied elementwise.

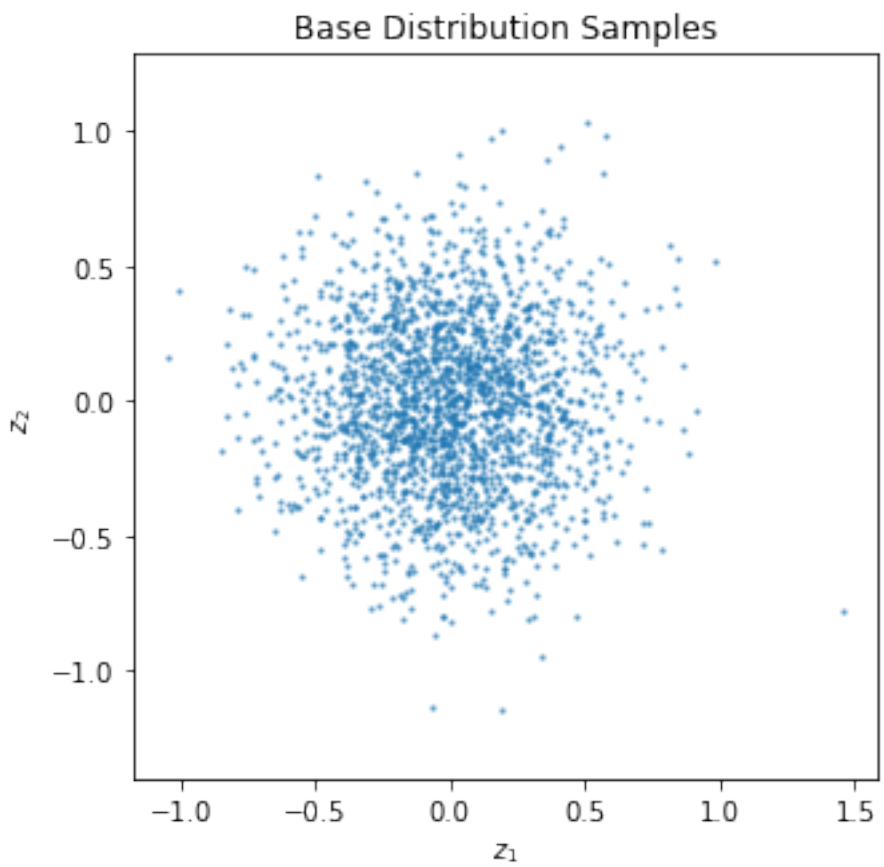
The transformed random variable x is given by $x = f_5(f_4(f_3(f_2(f_1(z)))))$. * You should use or construct bijectors for each of the transformations f_i, $i = 1, \ldots, 5$, and use `tfb.Chain` and `tfb.TransformedDistribution` to construct the final transformed distribution. * Ensure to implement the `log_det_jacobian` methods for any subclassed bijectors that you write. * Display a scatter plot of samples from the base distribution. * Display 4 scatter plot images of the transformed distribution from your random normalising flow, using samples of θ and a. Fix the axes of these 4 plots to the range $[-1, 1]$.

```
In [11]: class QuadraticBijector(tfb.Bijector):
             """f3(z) = (z1, z2 + a * z1^2)"""
             def __init__(self, a, **kwargs):
                 super().__init__(forward_min_event_ndims=1, name='quadratic', **kwargs)
                 self.a = tf.cast(a, tf.float32)

             def _forward(self, z):
```

```
        z1, z2 = z[..., :1], z[..., 1:]
        return tf.concat([z1, z2 + self.a * z1**2], axis=-1)

    def _inverse(self, y):
        y1, y2 = y[..., :1], y[..., 1:]
        return tf.concat([y1, y2 - self.a * y1**2], axis=-1)

    def _forward_log_det_jacobian(self, z):
        return tf.zeros_like(z[..., 0])

    def _inverse_log_det_jacobian(self, y):
        return tf.zeros_like(y[..., 0])


class RotationBijector(tfb.Bijector):
    """f4(z) = R @ z, where R is a 2D rotation matrix with angle theta"""
    def __init__(self, theta, **kwargs):
        super().__init__(forward_min_event_ndims=1, name='rotation', **kwargs)
        self.theta = tf.cast(theta, tf.float32)

    def _forward(self, z):
        c, s = tf.cos(self.theta), tf.sin(self.theta)
        z1, z2 = z[..., :1], z[..., 1:]
        return tf.concat([c * z1 - s * z2, s * z1 + c * z2], axis=-1)

    def _inverse(self, y):
        c, s = tf.cos(self.theta), tf.sin(self.theta)
        y1, y2 = y[..., :1], y[..., 1:]
        return tf.concat([c * y1 + s * y2, -s * y1 + c * y2], axis=-1)

    def _forward_log_det_jacobian(self, z):
        return tf.zeros_like(z[..., 0])

    def _inverse_log_det_jacobian(self, y):
        return tf.zeros_like(y[..., 0])
```

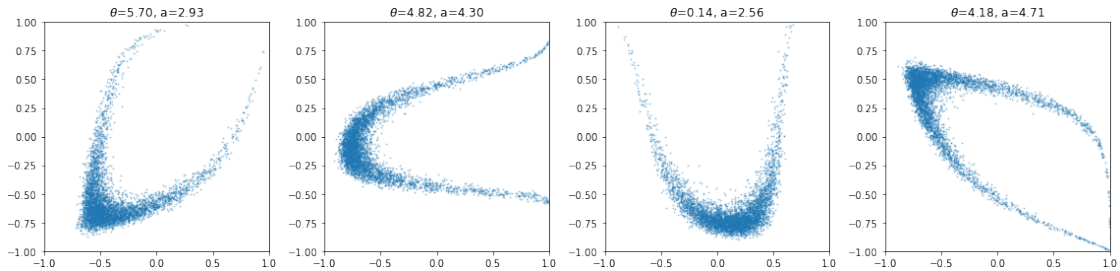
In [12]:
```
base_dist = tfd.MultivariateNormalDiag(loc=[0., 0.], scale_diag=[0.3, 0.3])

def make_transformed_dist(theta, a):
    """Build the normalising flow: x = f5(f4(f3(f2(f1(z)))))"""
    bijector = tfb.Chain([
        tfb.Tanh(),                     # f5: tanh elementwise
        RotationBijector(theta),         # f4: rotation by theta
        QuadraticBijector(a),            # f3: (z1, z2 + a*z1^2)
        tfb.Scale([1., 0.5]),            # f2: (z1, z2/2)
        tfb.Shift([0., -2.]),            # f1: (z1, z2 - 2)
    ])
    return tfd.TransformedDistribution(distribution=base_dist, bijector=bijector)
```

```
In [13]: samples = base_dist.sample(2000).numpy()
         plt.figure(figsize=(5, 5))
         plt.scatter(samples[:, 0], samples[:, 1], s=2, alpha=0.5)
         plt.title('Base Distribution Samples')
         plt.xlabel('$z_1$')
         plt.ylabel('$z_2$')
         plt.axis('equal')
         plt.show()
```




```
In [14]: fig, axes = plt.subplots(1, 4, figsize=(16, 4))
         for i in range(4):
             theta_i = np.random.uniform(0, 2 * np.pi)
             a_i = np.random.normal(3, 1)
             td = make_transformed_dist(theta_i, a_i)
             samps = td.sample(5000).numpy()
             axes[i].scatter(samps[:, 0], samps[:, 1], s=1, alpha=0.3)
             axes[i].set_xlim([-1, 1])
             axes[i].set_ylim([-1, 1])
             axes[i].set_title(f'$\\theta$={theta_i:.2f}, a={a_i:.2f}')
```

```
plt.tight_layout()
plt.show()
```



In [ ]:

In [ ]:

1.2 2. Create the image dataset

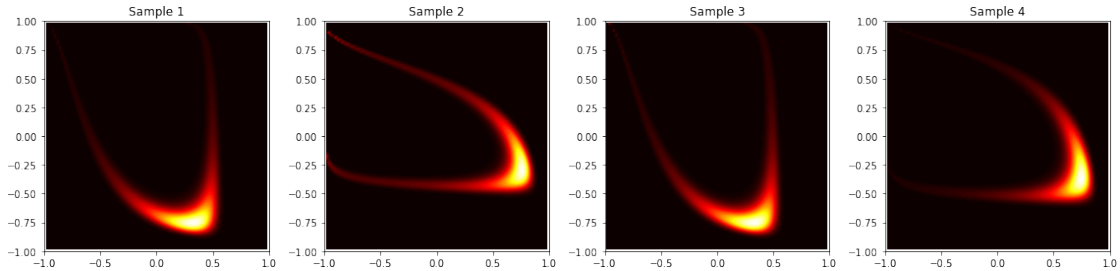
- You should now use your random normalising flow to generate an image dataset of contour plots from your random normalising flow network.
  - Feel free to get creative and experiment with different architectures to produce different sets of images!
- First, display a sample of 4 contour plot images from your normalising flow network using 4 independently sampled sets of parameters.
  - You may find the following `get_densities` function useful: this calculates density values for a (batched) Distribution for use in a contour plot.
- Your dataset should consist of at least 1000 images, stored in a numpy array of shape `(N, 36, 36, 3)`. Each image in the dataset should correspond to a contour plot of a transformed distribution from a normalising flow with an independently sampled set of parameters s, T, S, b. It will take a few minutes to create the dataset.
- As well as the `get_densities` function, the `get_image_array_from_density_values` function will help you to generate the dataset.
  - This function creates a numpy array for an image of the contour plot for a given set of density values Z. Feel free to choose your own options for the contour plots.
- Display a sample of 20 images from your generated dataset in a figure.

In [15]:
```
# Helper function to compute transformed distribution densities

X, Y = np.meshgrid(np.linspace(-1, 1, 100), np.linspace(-1, 1, 100))
inputs = np.transpose(np.stack((X, Y)), [1, 2, 0])

def get_densities(transformed_distribution):
    """
```

```
    This function takes a (batched) Distribution object as an argument, and returns a
    array Z of shape (batch_shape, 100, 100) of density values, that can be used to m
    contour plot with:
    plt.contourf(X, Y, Z[b, ...], cmap='hot', levels=100)
    where b is an index into the batch shape.
    """
    batch_shape = transformed_distribution.batch_shape
    Z = transformed_distribution.prob(np.expand_dims(inputs, 2))
    Z = np.transpose(Z, list(range(2, 2+len(batch_shape))) + [0, 1])
    return Z
```

In [16]:
```
# Helper function to convert contour plots to numpy arrays

import numpy as np
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
from matplotlib.figure import Figure

def get_image_array_from_density_values(Z):
    """
    This function takes a numpy array Z of density values of shape (100, 100)
    and returns an integer numpy array of shape (36, 36, 3) of pixel values for an im
    """
    assert Z.shape == (100, 100)
    fig = Figure(figsize=(0.5, 0.5))
    canvas = FigureCanvas(fig)
    ax = fig.gca()
    ax.contourf(X, Y, Z, cmap='hot', levels=100)
    ax.axis('off')
    fig.tight_layout(pad=0)

    ax.margins(0)
    fig.canvas.draw()
    image_from_plot = np.frombuffer(fig.canvas.tostring_rgb(), dtype=np.uint8)
    image_from_plot = image_from_plot.reshape(fig.canvas.get_width_height()[::-1] + (3
    return image_from_plot
```

In [17]:
```
inputs_f32 = inputs.astype(np.float32)

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for i in range(4):
    theta_i = np.random.uniform(0, 2 * np.pi)
    a_i = np.random.normal(3, 1)
    td = make_transformed_dist(theta_i, a_i)
    Z = td.prob(inputs_f32).numpy()
    axes[i].contourf(X, Y, Z, cmap='hot', levels=100)
    axes[i].set_title(f'Sample {i+1}')
plt.tight_layout()
plt.show()
```

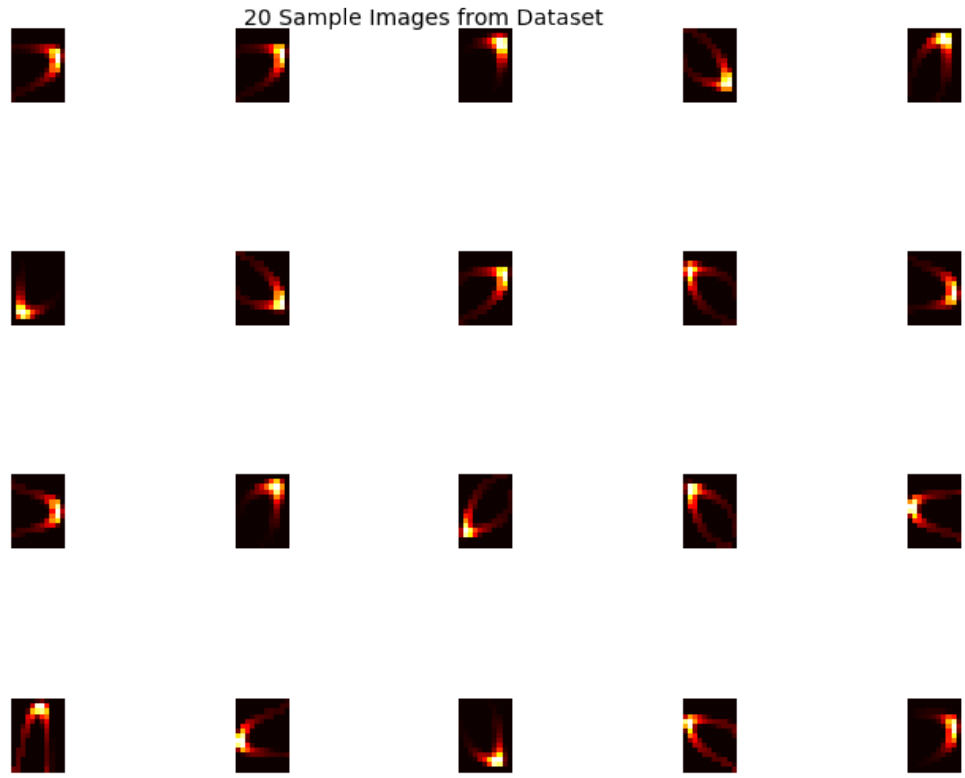



```
In [18]: # Determine actual image shape from a test image
         theta_test = np.random.uniform(0, 2 * np.pi)
         a_test = np.random.normal(3, 1)
         td_test = make_transformed_dist(theta_test, a_test)
         Z_test = td_test.prob(inputs_f32).numpy()
         test_img = get_image_array_from_density_values(Z_test)
         img_h, img_w, img_c = test_img.shape
         print(f'Image shape: ({img_h}, {img_w}, {img_c})')

         N = 1000
         dataset_images = np.zeros((N, img_h, img_w, img_c), dtype=np.uint8)

         for i in range(N):
             theta_i = np.random.uniform(0, 2 * np.pi)
             a_i = np.random.normal(3, 1)
             td = make_transformed_dist(theta_i, a_i)
             Z = td.prob(inputs_f32).numpy()
             dataset_images[i] = get_image_array_from_density_values(Z)
             if (i + 1) % 200 == 0:
                 print(f'Generated {i+1}/{N} images')

         print(f'Dataset shape: {dataset_images.shape}')
```

```
Image shape: (36, 36, 3)
Generated 200/1000 images
Generated 400/1000 images
Generated 600/1000 images
Generated 800/1000 images
Generated 1000/1000 images
Dataset shape: (1000, 36, 36, 3)
```

```
In [19]: fig, axes = plt.subplots(4, 5, figsize=(10, 8))
         indices = np.random.choice(N, 20, replace=False)
         for i, idx in enumerate(indices):
             ax = axes[i // 5, i % 5]
             ax.imshow(dataset_images[idx])
```

```
    ax.axis('off')
plt.suptitle('20 Sample Images from Dataset', fontsize=14)
plt.tight_layout()
plt.show()
```



In [ ]:

In [ ]:

In [ ]:

1.3 3. Make `tf.data.Dataset` objects

- You should now split your dataset to create `tf.data.Dataset` objects for training and validation data.
- Using the `map` method, normalise the pixel values so that they lie between 0 and 1.
- These Datasets will be used to train a variational autoencoder (VAE). Use the `map` method to return a tuple of input and output Tensors where the image is duplicated as both input and output.
- Randomly shuffle the training Dataset.

- Batch both datasets with a batch size of 20, setting `drop_remainder=True`.
- Print the `element_spec` property for one of the Dataset objects.

```
In [20]: train_size = int(0.8 * N)
         train_images = dataset_images[:train_size]
         val_images = dataset_images[train_size:]

         train_ds = tf.data.Dataset.from_tensor_slices(train_images)
         val_ds = tf.data.Dataset.from_tensor_slices(val_images)

         def preprocess(image):
             image = tf.cast(image, tf.float32) / 255.0
             return (image, image)

         train_ds = train_ds.map(preprocess).shuffle(buffer_size=train_size).batch(20, drop_rem
         val_ds = val_ds.map(preprocess).batch(20, drop_remainder=True)

         print('Training Dataset element_spec:')
         print(train_ds.element_spec)
         print(f'\nTraining batches: {tf.data.experimental.cardinality(train_ds).numpy()}')
         print(f'Validation batches: {tf.data.experimental.cardinality(val_ds).numpy()}')
```

```
Training Dataset element_spec:
(TensorSpec(shape=(20, 36, 36, 3), dtype=tf.float32, name=None), TensorSpec(shape=(20, 36, 36,

Training batches: 40
Validation batches: 10
```

```
In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:
```

1.4 4. Build the encoder and decoder networks

- You should now create the encoder and decoder for the variational autoencoder algorithm.
- You should design these networks yourself, subject to the following constraints:
  - The encoder and decoder networks should be built using the `Sequential` class.
  - The encoder and decoder networks should use probabilistic layers where necessary to represent distributions.
  - The prior distribution should be a zero-mean, isotropic Gaussian (identity covariance matrix).
  - The encoder network should add the KL divergence loss to the model.

- Print the model summary for the encoder and decoder networks.

```
In [21]: latent_size = 2

         prior = tfd.MultivariateNormalDiag(
             loc=tf.zeros(latent_size),
             scale_diag=tf.ones(latent_size)
         )

         event_size = img_h * img_w * img_c

         encoder = tf.keras.Sequential([
             tf.keras.layers.InputLayer(input_shape=(img_h, img_w, img_c)),
             tf.keras.layers.Flatten(),
             tf.keras.layers.Dense(512, activation='relu'),
             tf.keras.layers.Dense(256, activation='relu'),
             tf.keras.layers.Dense(tfpl.MultivariateNormalTriL.params_size(latent_size)),
             tfpl.MultivariateNormalTriL(
                 latent_size,
                 activity_regularizer=tfpl.KLDivergenceRegularizer(prior, weight=1.0)
             )
         ])

         encoder.summary()
```

```
WARNING:tensorflow:From /opt/conda/lib/python3.7/site-packages/tensorflow_core/python/ops/lina
Instructions for updating:
Do not pass `graph_parents`.  They will  no longer be used.
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 3888)              0
_________________________________________________________________
dense (Dense)                (None, 512)               1991168
_________________________________________________________________
dense_1 (Dense)              (None, 256)               131328
_________________________________________________________________
dense_2 (Dense)              (None, 5)                 1285
_________________________________________________________________
multivariate_normal_tri_l (M ((None, 2), (None, 2))    0
=================================================================
Total params: 2,123,781
Trainable params: 2,123,781
Non-trainable params: 0
_________________________________________________________________
```

```
In [22]: decoder = tf.keras.Sequential([
```

```
            tf.keras.layers.InputLayer(input_shape=(latent_size,)),
            tf.keras.layers.Dense(256, activation='relu'),
            tf.keras.layers.Dense(512, activation='relu'),
            tf.keras.layers.Dense(tfpl.IndependentBernoulli.params_size(event_size)),
            tfpl.IndependentBernoulli(
                event_shape=(img_h, img_w, img_c),
                convert_to_tensor_fn=tfd.Distribution.mean
            )
        ])

        decoder.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_3 (Dense)              (None, 256)               768
_________________________________________________________________
dense_4 (Dense)              (None, 512)               131584
_________________________________________________________________
dense_5 (Dense)              (None, 3888)              1994544
_________________________________________________________________
independent_bernoulli (Indep ((None, 36, 36, 3), (None 0
=================================================================
Total params: 2,126,896
Trainable params: 2,126,896
Non-trainable params: 0
_________________________________________________________________
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

1.5 5. Train the variational autoencoder

- You should now train the variational autoencoder. Build the VAE using the `Model` class and the encoder and decoder models. Print the model summary.
- Compile the VAE with the negative log likelihood loss and train with the `fit` method, using the training and validation Datasets.
- Plot the learning curves for loss vs epoch for both training and validation sets.

In [23]:
```
x_input = tf.keras.layers.Input(shape=(img_h, img_w, img_c))
z = encoder(x_input)
x_recon = decoder(z)
```

```
vae = tf.keras.Model(inputs=x_input, outputs=x_recon)

vae.summary()
```

```
Model: "model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_3 (InputLayer)         [(None, 36, 36, 3)]       0
_________________________________________________________________
sequential (Sequential)      (None, 2)                 2123781
_________________________________________________________________
sequential_1 (Sequential)    (None, 36, 36, 3)         2126896
=================================================================
Total params: 4,250,677
Trainable params: 4,250,677
Non-trainable params: 0
_________________________________________________________________
```

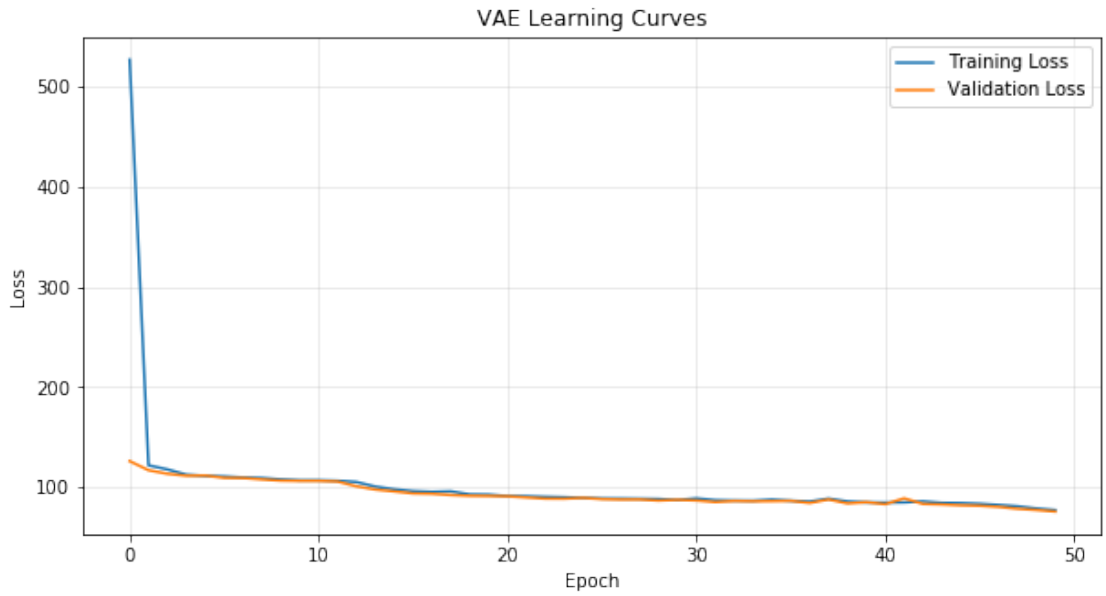
In [24]:
```
nll = lambda y_true, y_pred: -y_pred.log_prob(y_true)

vae.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3), loss=nll)

history = vae.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    verbose=1
)
```

```
Train for 40 steps, validate for 10 steps
Epoch 1/50
40/40 [==============================] - 10s 257ms/step - loss: 527.2704 - val_loss: 125.9860
Epoch 2/50
40/40 [==============================] - 8s 192ms/step - loss: 121.7750 - val_loss: 116.8673
Epoch 3/50
40/40 [==============================] - 8s 192ms/step - loss: 117.6852 - val_loss: 113.2959
Epoch 4/50
40/40 [==============================] - 8s 193ms/step - loss: 112.4478 - val_loss: 111.3491
Epoch 5/50
40/40 [==============================] - 7s 187ms/step - loss: 111.2203 - val_loss: 111.5676
Epoch 6/50
40/40 [==============================] - 8s 188ms/step - loss: 110.6456 - val_loss: 109.1995
Epoch 7/50
40/40 [==============================] - 7s 187ms/step - loss: 109.5734 - val_loss: 109.0586
Epoch 8/50
40/40 [==============================] - 7s 187ms/step - loss: 109.0684 - val_loss: 107.6867
```

```
Epoch 9/50
40/40 [==============================] - 7s 185ms/step - loss: 107.6521 - val_loss: 106.4828
Epoch 10/50
40/40 [==============================] - 8s 188ms/step - loss: 106.8014 - val_loss: 106.2237
Epoch 11/50
40/40 [==============================] - 8s 192ms/step - loss: 106.7857 - val_loss: 106.2163
Epoch 12/50
40/40 [==============================] - 8s 190ms/step - loss: 106.3194 - val_loss: 105.6286
Epoch 13/50
40/40 [==============================] - 7s 185ms/step - loss: 105.1060 - val_loss: 100.7991
Epoch 14/50
40/40 [==============================] - 8s 188ms/step - loss: 100.5277 - val_loss: 97.5556
Epoch 15/50
40/40 [==============================] - 7s 185ms/step - loss: 97.7667 - val_loss: 95.7185
Epoch 16/50
40/40 [==============================] - 8s 190ms/step - loss: 95.9774 - val_loss: 93.7008
Epoch 17/50
40/40 [==============================] - 8s 192ms/step - loss: 95.2426 - val_loss: 93.3700
Epoch 18/50
40/40 [==============================] - 8s 197ms/step - loss: 95.7281 - val_loss: 92.0830
Epoch 19/50
40/40 [==============================] - 8s 198ms/step - loss: 92.7049 - val_loss: 91.2948
Epoch 20/50
40/40 [==============================] - 8s 195ms/step - loss: 92.4849 - val_loss: 91.1104
Epoch 21/50
40/40 [==============================] - 7s 183ms/step - loss: 91.0263 - val_loss: 90.8328
Epoch 22/50
40/40 [==============================] - 7s 185ms/step - loss: 90.8745 - val_loss: 89.6386
Epoch 23/50
40/40 [==============================] - 7s 185ms/step - loss: 90.2840 - val_loss: 88.4725
Epoch 24/50
40/40 [==============================] - 7s 187ms/step - loss: 89.8696 - val_loss: 88.3870
Epoch 25/50
40/40 [==============================] - 7s 183ms/step - loss: 89.2360 - val_loss: 89.2363
Epoch 26/50
40/40 [==============================] - 8s 192ms/step - loss: 88.6133 - val_loss: 87.9677
Epoch 27/50
40/40 [==============================] - 7s 183ms/step - loss: 88.5387 - val_loss: 87.5791
Epoch 28/50
40/40 [==============================] - 7s 183ms/step - loss: 88.3347 - val_loss: 87.6179
Epoch 29/50
40/40 [==============================] - 8s 195ms/step - loss: 88.0985 - val_loss: 86.5170
Epoch 30/50
40/40 [==============================] - 7s 180ms/step - loss: 87.0828 - val_loss: 87.2799
Epoch 31/50
40/40 [==============================] - 7s 182ms/step - loss: 88.7033 - val_loss: 86.6152
Epoch 32/50
40/40 [==============================] - 7s 185ms/step - loss: 86.9521 - val_loss: 85.1077
```

```
Epoch 33/50
40/40 [==============================] - 8s 195ms/step - loss: 86.5221 - val_loss: 86.1170
Epoch 34/50
40/40 [==============================] - 8s 190ms/step - loss: 86.3812 - val_loss: 85.8314
Epoch 35/50
40/40 [==============================] - 8s 190ms/step - loss: 87.4863 - val_loss: 85.9878
Epoch 36/50
40/40 [==============================] - 8s 188ms/step - loss: 86.3755 - val_loss: 86.2426
Epoch 37/50
40/40 [==============================] - 7s 185ms/step - loss: 85.5355 - val_loss: 84.1543
Epoch 38/50
40/40 [==============================] - 7s 185ms/step - loss: 88.4727 - val_loss: 87.6003
Epoch 39/50
40/40 [==============================] - 7s 185ms/step - loss: 85.6587 - val_loss: 83.7126
Epoch 40/50
40/40 [==============================] - 8s 193ms/step - loss: 84.6960 - val_loss: 84.9295
Epoch 41/50
40/40 [==============================] - 8s 190ms/step - loss: 84.5162 - val_loss: 82.9323
Epoch 42/50
40/40 [==============================] - 8s 188ms/step - loss: 84.7049 - val_loss: 88.7249
Epoch 43/50
40/40 [==============================] - 8s 190ms/step - loss: 85.6838 - val_loss: 83.1841
Epoch 44/50
40/40 [==============================] - 7s 185ms/step - loss: 84.2250 - val_loss: 82.6838
Epoch 45/50
40/40 [==============================] - 7s 185ms/step - loss: 83.9647 - val_loss: 81.8053
Epoch 46/50
40/40 [==============================] - 7s 180ms/step - loss: 83.3146 - val_loss: 81.3602
Epoch 47/50
40/40 [==============================] - 7s 183ms/step - loss: 82.1320 - val_loss: 80.2248
Epoch 48/50
40/40 [==============================] - 8s 188ms/step - loss: 80.8115 - val_loss: 78.3434
Epoch 49/50
40/40 [==============================] - 7s 183ms/step - loss: 78.6124 - val_loss: 77.1274
Epoch 50/50
40/40 [==============================] - 7s 177ms/step - loss: 77.0061 - val_loss: 75.5583
```

In [25]:
```
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('VAE Learning Curves')
plt.grid(True, alpha=0.3)
plt.show()
```



In [ ]:

In [ ]:

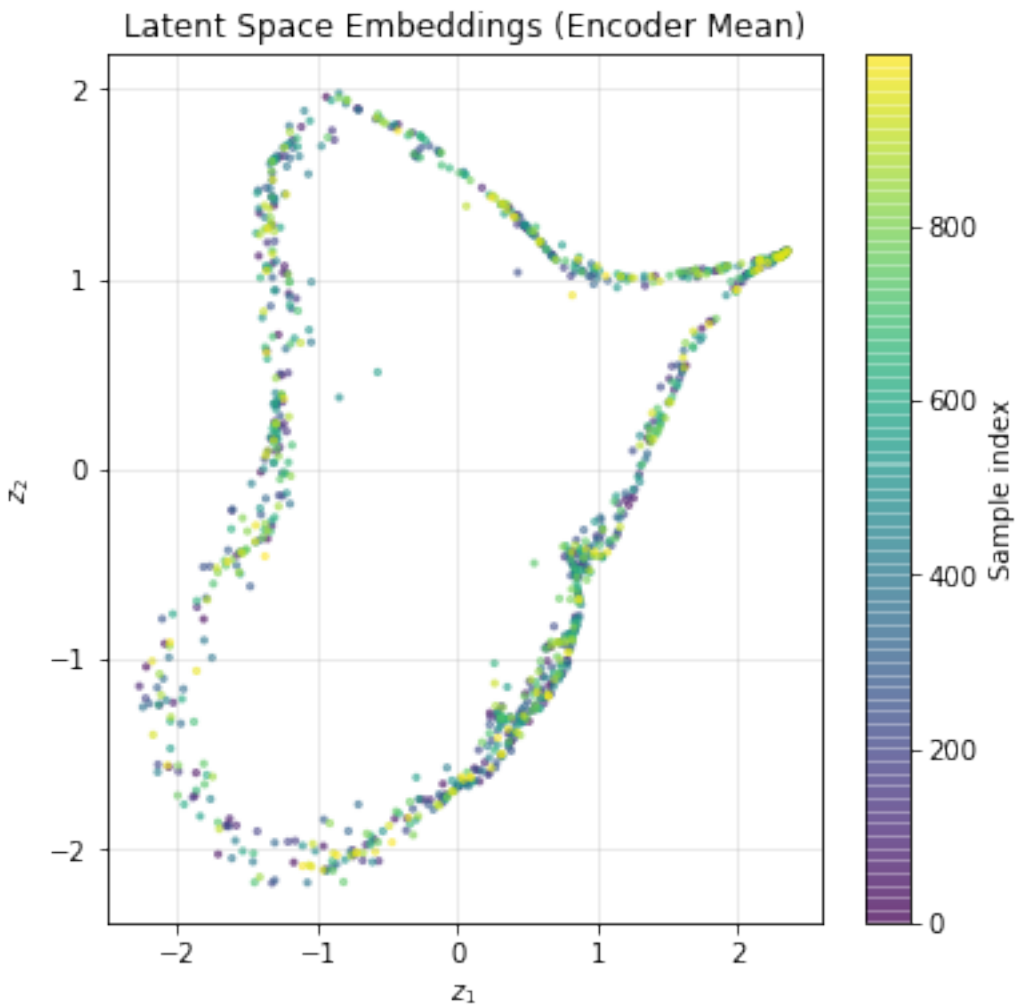
In [ ]:

1.6 6. Use the encoder and decoder networks

- You can now put your encoder and decoder networks into practice!
- Randomly sample 1000 images from the dataset, and pass them through the encoder. Display the embeddings in a scatter plot (project to 2 dimensions if the latent space has dimension higher than two).
- Randomly sample 4 images from the dataset and for each image, display the original and reconstructed image from the VAE in a figure.
  - Use the mean of the output distribution to display the images.
- Randomly sample 6 latent variable realisations from the prior distribution, and display the images in a figure.
  - Again use the mean of the output distribution to display the images.

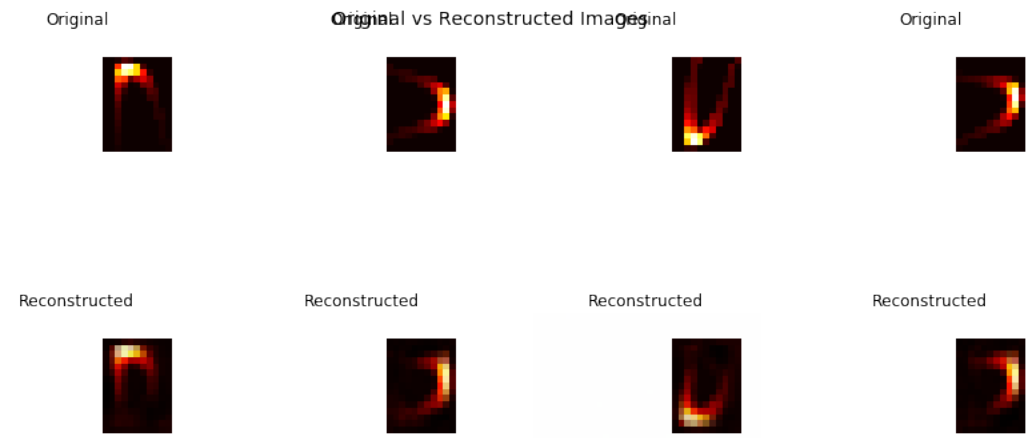
In [26]:
```
sample_indices = np.random.choice(len(dataset_images), 1000, replace=False)
sample_imgs = dataset_images[sample_indices].astype(np.float32) / 255.0

z_dist = encoder(sample_imgs)
z_mean = z_dist.mean().numpy()

plt.figure(figsize=(6, 6))
```

```
plt.scatter(z_mean[:, 0], z_mean[:, 1], s=5, alpha=0.5, c=np.arange(1000), cmap='viri
plt.colorbar(label='Sample index')
plt.xlabel('$z_1$')
plt.ylabel('$z_2$')
plt.title('Latent Space Embeddings (Encoder Mean)')
plt.grid(True, alpha=0.3)
plt.show()
```




```
In [27]: recon_indices = np.random.choice(len(dataset_images), 4, replace=False)
         recon_imgs = dataset_images[recon_indices].astype(np.float32) / 255.0

         recon_output = vae(recon_imgs)
         recon_mean = recon_output.mean().numpy()

         fig, axes = plt.subplots(2, 4, figsize=(12, 6))
```

```
for i in range(4):
    axes[0, i].imshow(recon_imgs[i])
    axes[0, i].set_title('Original')
    axes[0, i].axis('off')
    axes[1, i].imshow(np.clip(recon_mean[i], 0, 1))
    axes[1, i].set_title('Reconstructed')
    axes[1, i].axis('off')
plt.suptitle('Original vs Reconstructed Images', fontsize=14)
plt.tight_layout()
plt.show()
```



In [28]:
```
z_samples = prior.sample(6)
gen_output = decoder(z_samples)
gen_mean = gen_output.mean().numpy()

fig, axes = plt.subplots(1, 6, figsize=(18, 3))
for i in range(6):
    axes[i].imshow(np.clip(gen_mean[i], 0, 1))
    axes[i].axis('off')
    axes[i].set_title(f'Sample {i+1}')
plt.suptitle('Generated Images from Prior Samples', fontsize=14)
plt.tight_layout()
plt.show()
```

In [ ]:

In [ ]:

In [ ]:

1.7 Make a video of latent space interpolation (not assessed)

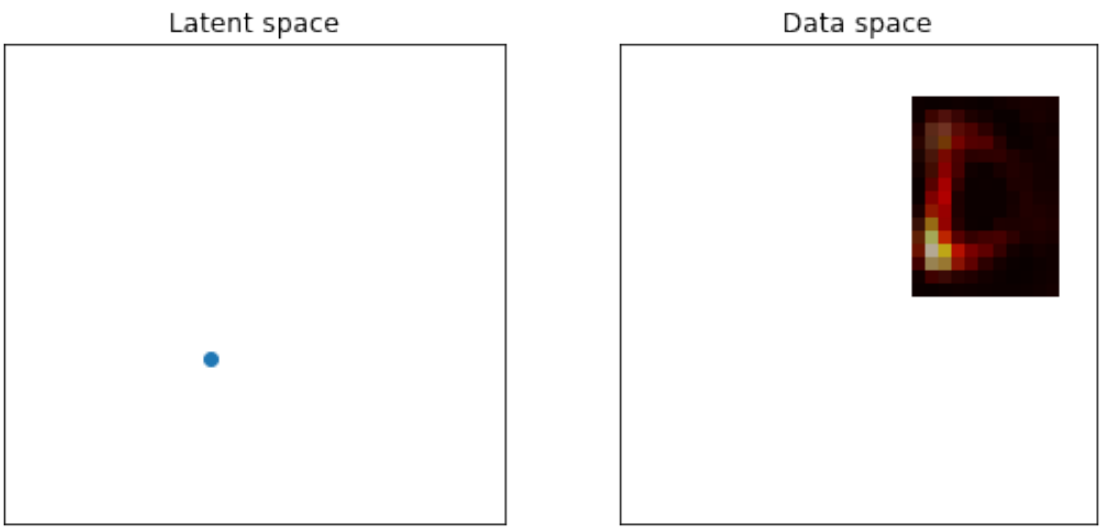
- Just for fun, you can run the code below to create a video of your decoder's generations, depending on the latent space.

In [29]:
```
# Function to create animation

import matplotlib.animation as anim
from IPython.display import HTML


def get_animation(latent_size, decoder, interpolation_length=500):
    assert latent_size >= 2, "Latent space must be at least 2-dimensional for plottin
    fig = plt.figure(figsize=(9, 4))
    ax1 = fig.add_subplot(1,2,1)
    ax1.set_xlim([-3, 3])
    ax1.set_ylim([-3, 3])
    ax1.set_title("Latent space")
    ax1.axes.get_xaxis().set_visible(False)
    ax1.axes.get_yaxis().set_visible(False)
    ax2 = fig.add_subplot(1,2,2)
    ax2.set_title("Data space")
    ax2.axes.get_xaxis().set_visible(False)
    ax2.axes.get_yaxis().set_visible(False)

    # initializing a line variable
    line, = ax1.plot([], [], marker='o')
    img2 = ax2.imshow(np.zeros((36, 36, 3)))

    freqs = np.random.uniform(low=0.1, high=0.2, size=(latent_size,))
    phases = np.random.randn(latent_size)
    input_points = np.arange(interpolation_length)
    latent_coords = []
    for i in range(latent_size):
        latent_coords.append(2 * np.sin((freqs[i]*input_points + phases[i])).astype(n

    def animate(i):
        z = tf.constant([coord[i] for coord in latent_coords])
        img_out = np.squeeze(decoder(z[np.newaxis, ...]).mean().numpy())
        line.set_data(z.numpy()[0], z.numpy()[1])
```

```
            img2.set_data(np.clip(img_out, 0, 1))
            return (line, img2)

        return anim.FuncAnimation(fig, animate, frames=interpolation_length,
                                  repeat=False, blit=True, interval=150)
```

In [30]:
```
# Create the animation

a = get_animation(latent_size, decoder, interpolation_length=200)
HTML(a.to_html5_video())
```

Out[30]: <IPython.core.display.HTML object>



In [ ]: